



GIT FUNDAMENTALS

Programming Concepts - Week 1
Presented by: Marco Tripodi

WHAT IS GIT?



Git is a distributed version control system.



Helps track changes in code



Enables collaboration among developers



WHAT IS A REPOSITORY?

A REPOSITORY
STORES PROJECT
FILES AND
HISTORY.

LOCAL: STORED
ON YOUR
MACHINE

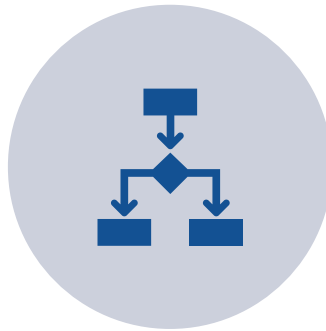
REMOTE: HOSTED
ONLINE (E.G.,
GITHUB, GITLAB)



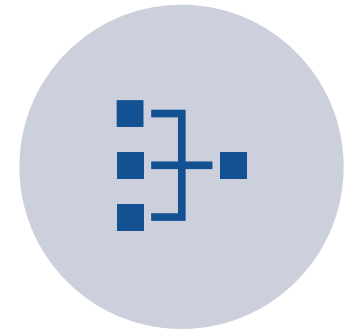
WHAT IS A BRANCH?



BRANCHES ARE SEPARATE
LINES OF DEVELOPMENT.



WORK ON MULTIPLE
FEATURES/FIXES
INDEPENDENTLY



MERGE BACK INTO MAIN
BRANCH



COMMON GIT COMMANDS



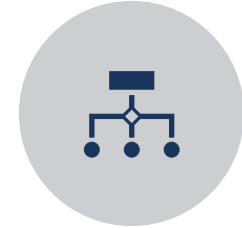
GIT
CHECKOUT



GIT PULL



GIT PUSH



GIT MERGE



GIT CHERRY-
PICK



GIT STASH



GIT FETCH



GIT COMMIT



GIT CHECKOUT



Switch between branches or restore files.



Example: git checkout feature-branch



GIT PULL



Fetch changes from remote repository and merge them into local branch.



Example: `git pull origin main`



GIT PUSH



Upload local commits to a remote repository.



Example: `git push origin main`



GIT MERGE



Combine changes from one branch into another.



Used to integrate feature branches into main branch



MERGE STRATEGIES



Fast-forward merge



Three-way merge



When to use each strategy



MERGE CONFLICTS



Occurs when changes clash between branches.



Must be resolved manually.



Tools: GUI & command line options



CONFLICT RESOLUTION PROCESS



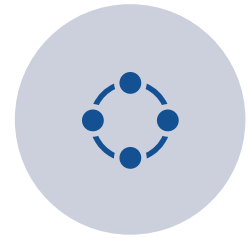
1. IDENTIFY
CONFLICTING FILES



2. EDIT FILES TO
FIX CONFLICTS



3. STAGE
RESOLVED FILES



4. COMMIT THE
MERGE



GIT CHERRY-PICK



Apply specific commits from one branch onto another.



Useful for selective bug fixes

GIT STASH



Temporarily save changes without committing.



Useful for switching tasks without losing work



GIT STASH



Download commits and data from remote repository.



Does not modify local branches

GIT COMMIT



Record changes to the repository.



Includes commit message describing changes



PULL REQUESTS (PRS)



PRs allow review and discussion before merging.



Common in Azure Dev Ops, GitHub, GitLab workflows



PULL REQUEST WORKFLOW



1. Create
feature
branch

2. Push
changes

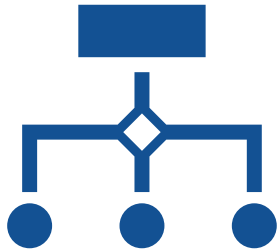
3. Open pull
request

4. Review &
approve

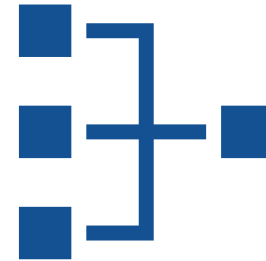
5. Merge into
main branch



VISUAL DIAGRAM: BRANCHING & MERGING



Branches help parallel development.



Merges integrate changes back into main.



BEST PRACTICES IN GIT

1

Commit often
with clear
messages

2

Use branches
for features

3

Regularly pull
changes to stay
updated



COLLABORATION IN GIT



Teams collaborate using Git hosting services like GitHub, GitLab, Bitbucket.



Supports code reviews and CI/CD.



GIT AND CI/CD



Git integrates with Continuous Integration/Continuous Deployment pipelines.



Automates testing and deployment.



BRANCHING MODELS

Git Flow

GitHub Flow

Trunk-Based
Development



TAGS IN GIT



Tags mark specific points in history (e.g., releases).



Example: `git tag v1.0.0`



UNDOING CHANGES



git reset

git revert

git checkout



INSPECTING HISTORY



git log

git diff

git blame



GIT GUI TOOLS

SourceTree

GitKraken

GitHub
Desktop



TROUBLESHOOTING IN GIT



Detached HEAD state



Merge conflicts



Push rejections



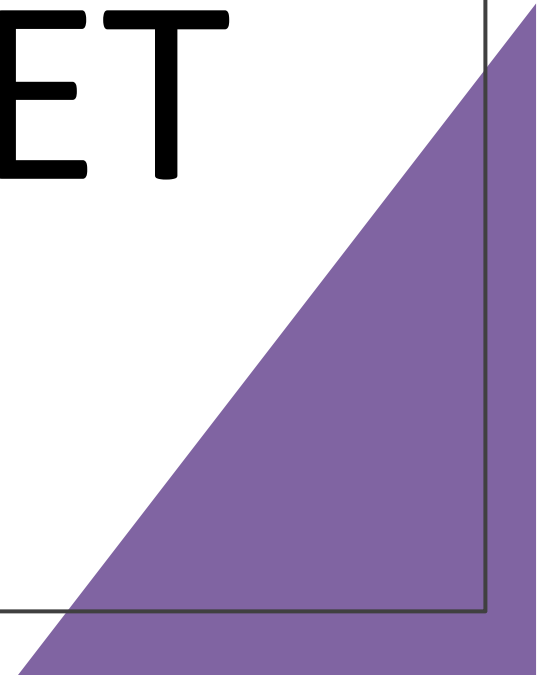
Solutions via rebase/reset



The image features a white background with decorative curved lines in the corners. In the top-left and bottom-left corners, there are light blue and orange curved lines. In the top-right and bottom-right corners, there are light blue and orange curved lines.

Q&A

Visual Studio 2022 & .NET



What is Visual Studio 2022?

- An IDE by Microsoft.
- Supports multiple languages
- Tools for coding, debugging, deployment

Visual Studio 2022 vs Visual Studio Code

Visual Studio 2022:
Full IDE, extensive
tools.

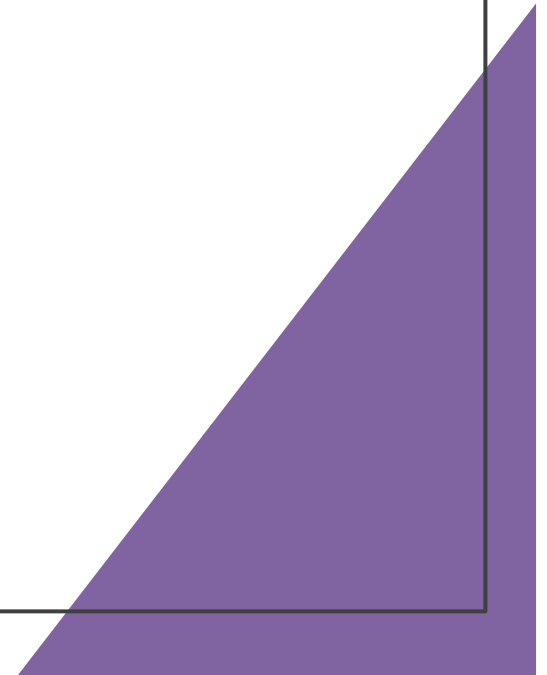
VS Code:
Lightweight editor,
flexible extensions.

Solution, Project, Class, Assembly

- Solution: Container for projects
- Project: Source code/resources
- Class: Blueprint for objects
- Assembly: Compiled DLL/EXE

Visual Studio 2022 Interface Overview

- Code editor
- Solution Explorer
- Properties window
- Output window
- Toolbar & menus



Creating a New Solution and Project

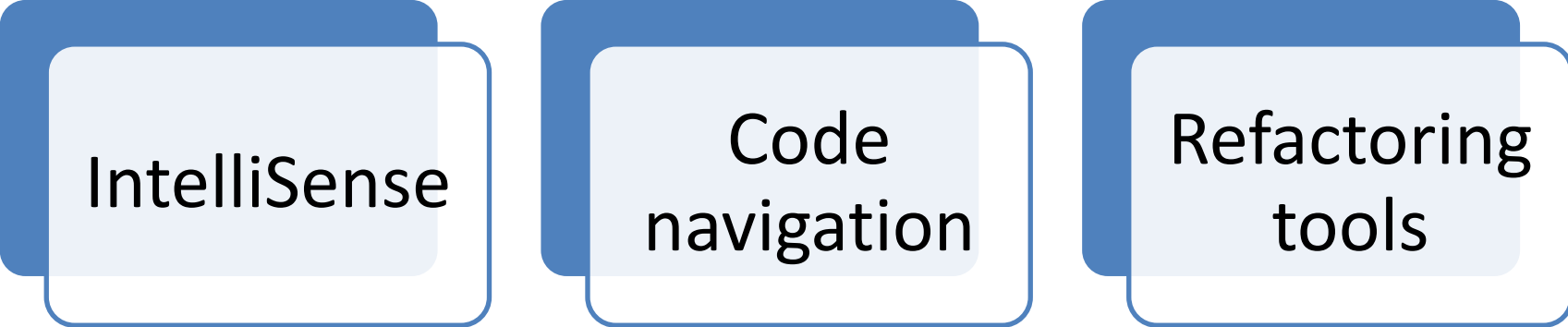
1. File > New Project

2. Choose Console
App (.NET)

3. Select
Framework/Core

4. Build & Run

Basic Features of Visual Studio



IntelliSense

Code
navigation

Refactoring
tools

Debugging Basics

Breakpoints

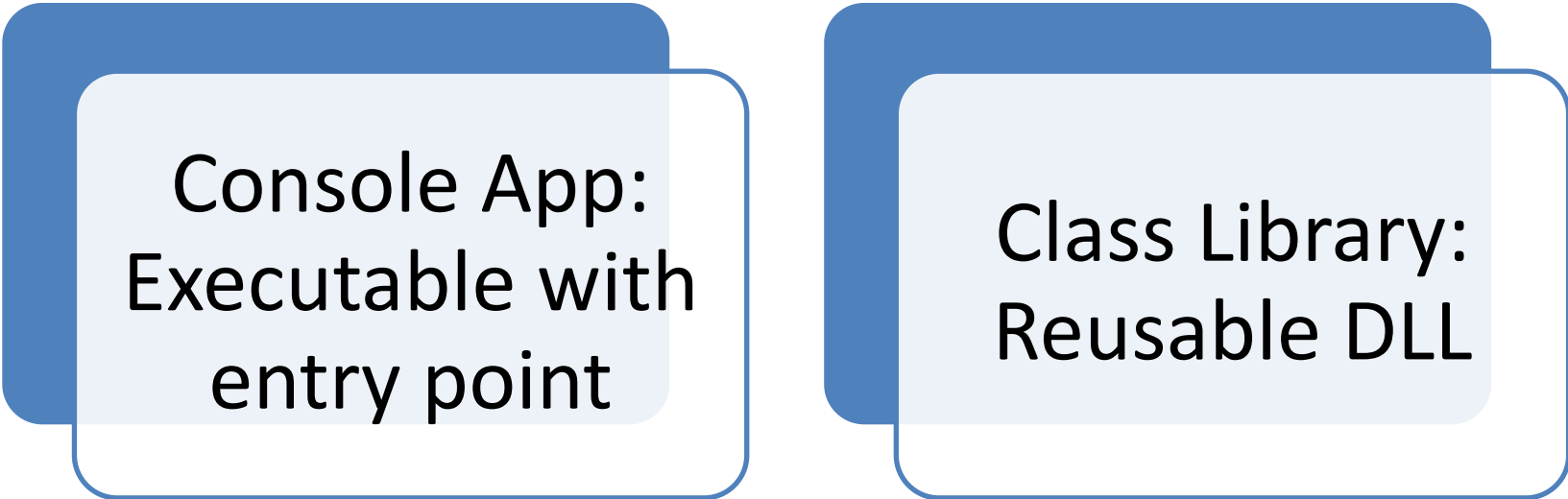
Step
over/into/out

Watch variables

Call stack
inspection



Console App vs Class Library



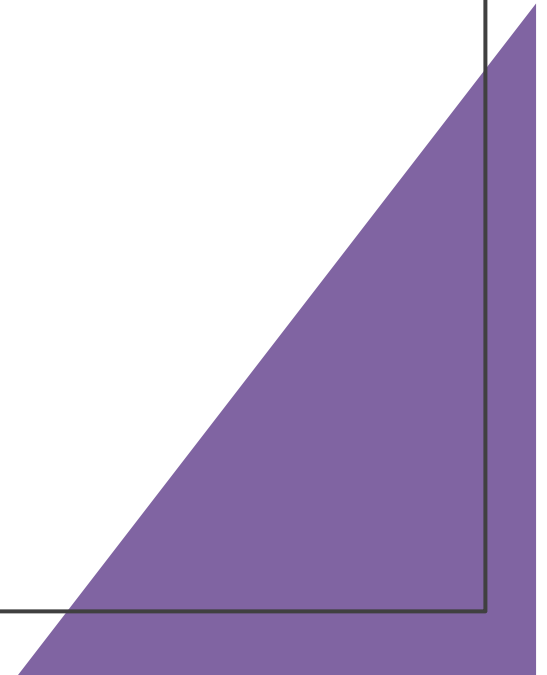
The diagram consists of two side-by-side rounded rectangular boxes. Each box has a dark blue header and a light blue body. The left box contains the text 'Console App: Executable with entry point'. The right box contains the text 'Class Library: Reusable DLL'. A horizontal purple line is at the bottom of the slide.

Console App:
Executable with
entry point

Class Library:
Reusable DLL

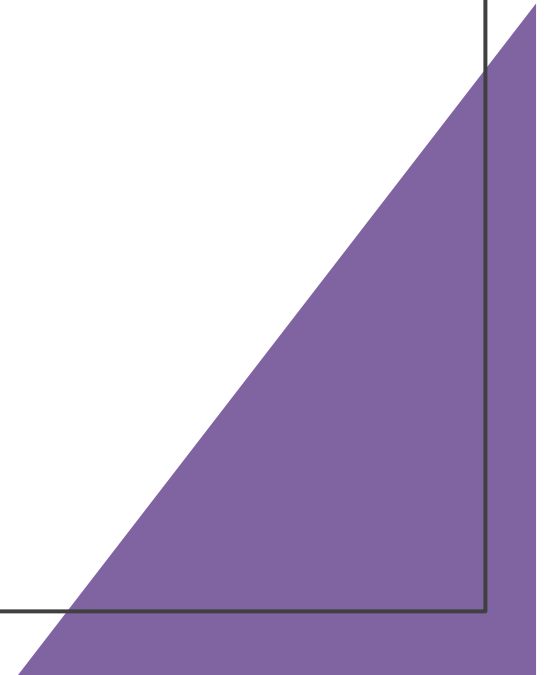
Managing Projects in a Solution

Add projects, set dependencies,
build/clean solutions.



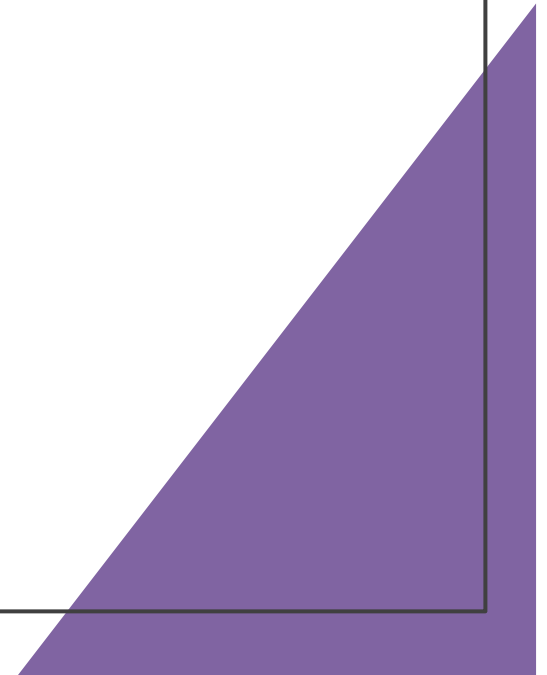
Git Integration in Visual Studio

- Built-in Git commands:
- Commit, Pull, Push
- Branch history view
- Merge conflict resolution



Using Git Features in Visual Studio

- Create/switch branches
- Resolve conflicts via GUI
- Open PRs from IDE



Code Snippets & Templates

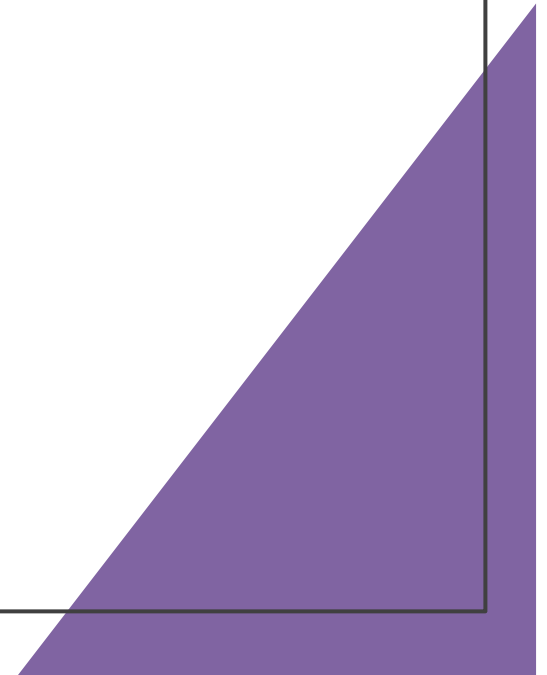
- Built-in snippets speed coding.
- You can create custom snippets too.

Tips & Tricks

- Keyboard shortcuts
- CodeLens insights
- Efficient navigation

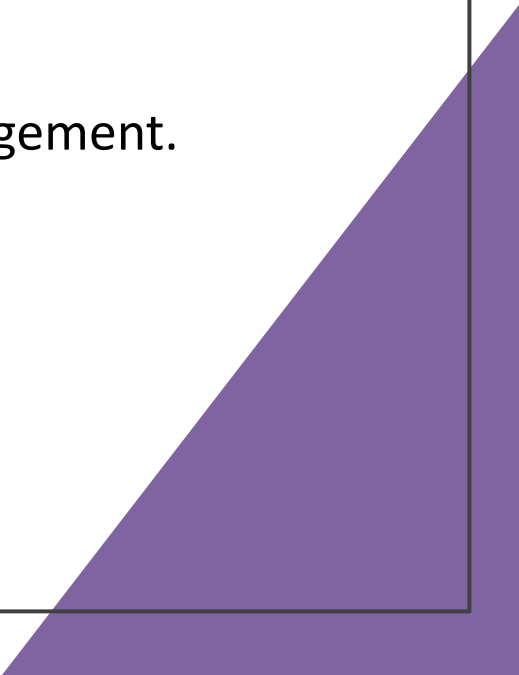
Building and Running Apps

- Build Solution
- Run with/without debugging
- Output/Errors window



NuGet Package Manager



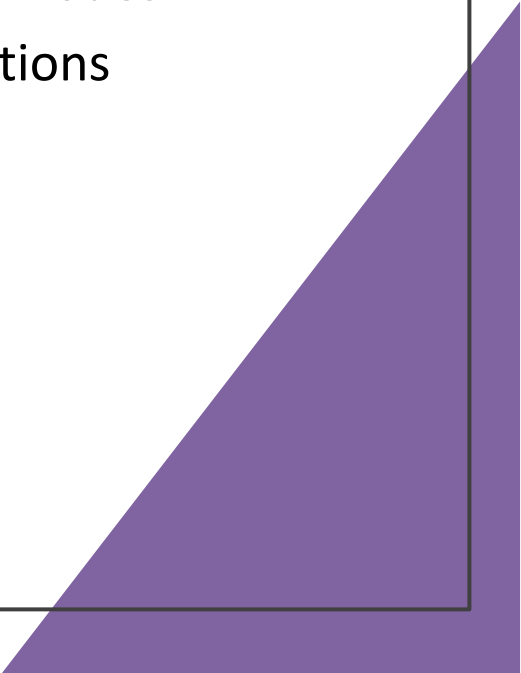
- Add/update/remove libraries.
 - NuGet simplifies dependency management.
- 

Error List and Output Windows

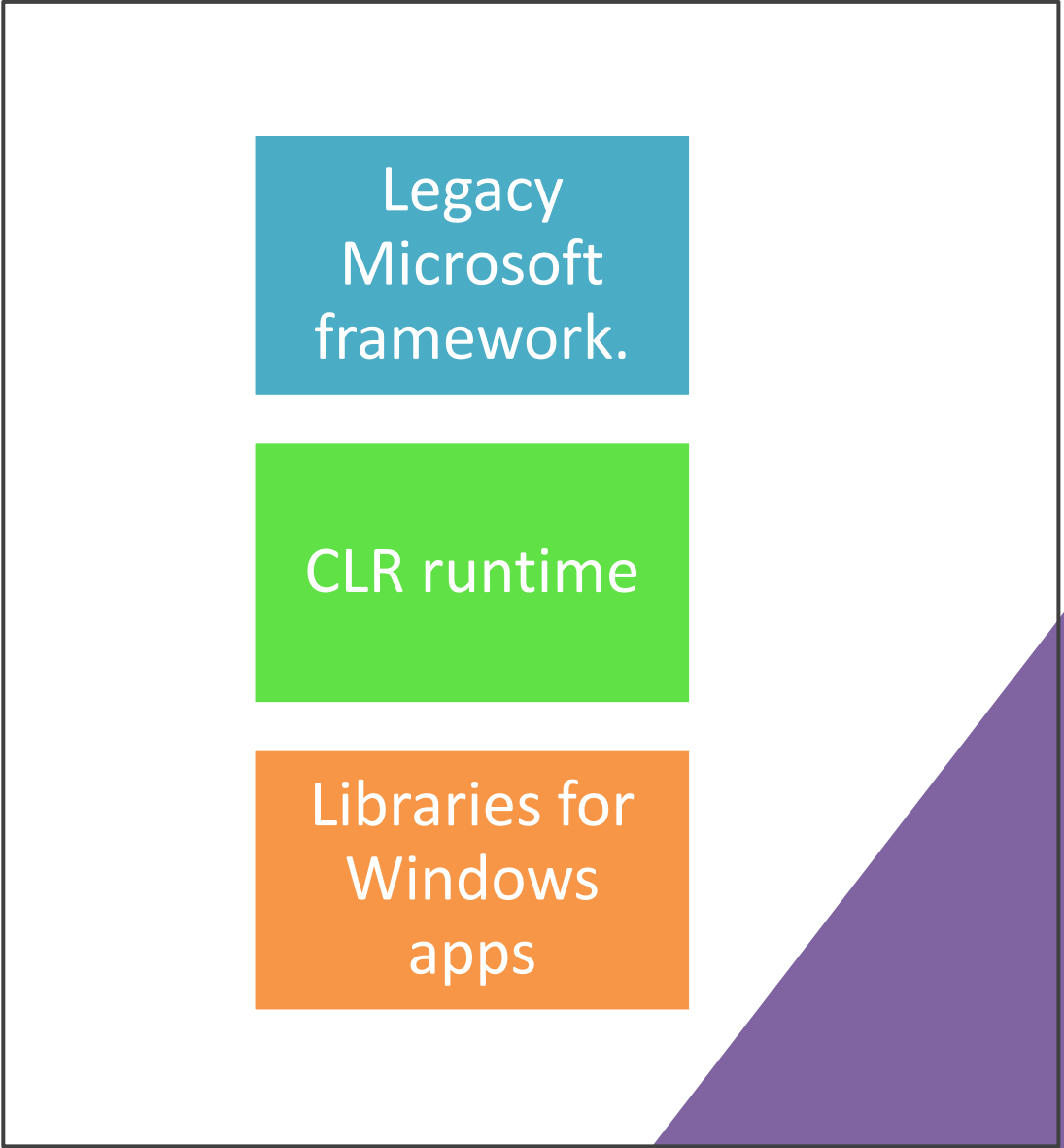
- Track errors, warnings, and build messages.
- Filter as needed.

Working with Configurations



- - Debug vs Release modes
 - - Custom configurations
- 

What is .NET Framework?



The diagram illustrates the layers of the .NET Framework. It consists of a large light gray rectangle with a thin black border. Inside this rectangle, three colored squares are stacked vertically: a blue square at the top, a green square in the middle, and an orange square at the bottom. To the right of the orange square, a purple triangle points upwards from the bottom right corner of the large rectangle. The text 'Legacy Microsoft framework.' is centered in the blue square, 'CLR runtime' is centered in the green square, and 'Libraries for Windows apps' is centered in the orange square.

Legacy
Microsoft
framework.

CLR runtime

Libraries for
Windows
apps

.NET Core Overview

Cross-platform,
open-source.

Runs on Windows,
Linux, macOS

Designed for cloud
& modern apps

Differences: .NET Framework vs .NET Core

.NET
Framework:
Windows only.

.NET Core:
Cross-platform,
open-source.

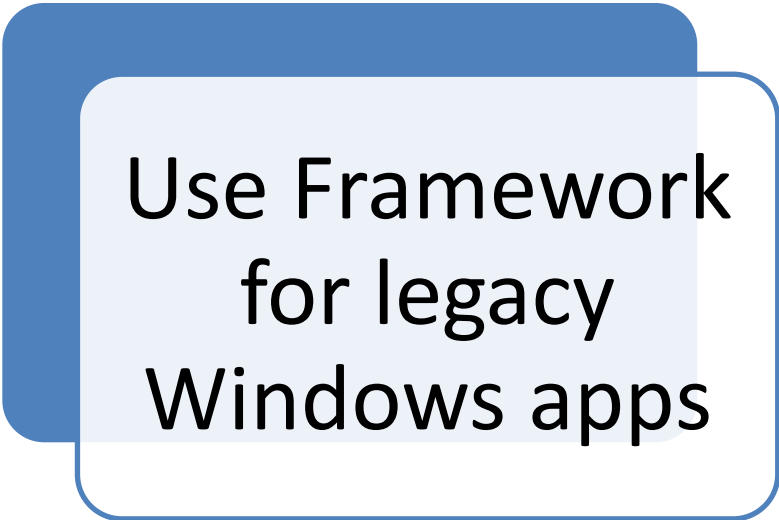
New apps:
Prefer Core/5+

.NET 5/6/7 and Beyond

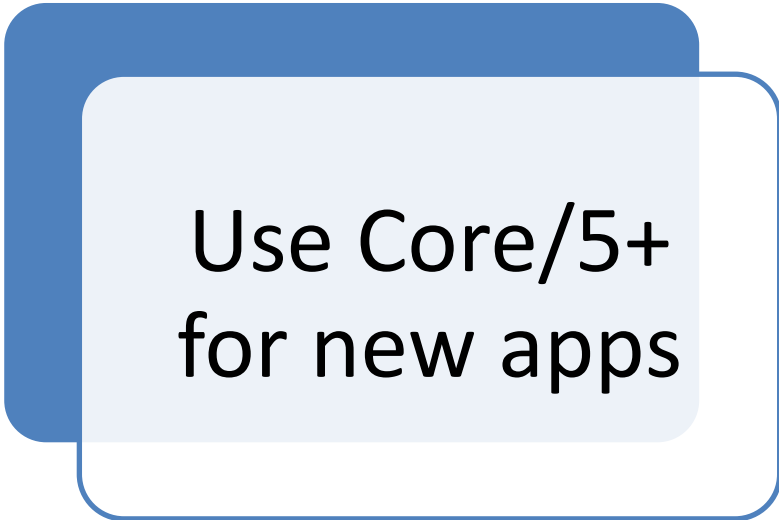
- Unified platform.
- Cross-platform
- High performance
- Frequent updates



Choosing Between .NET Framework & Core



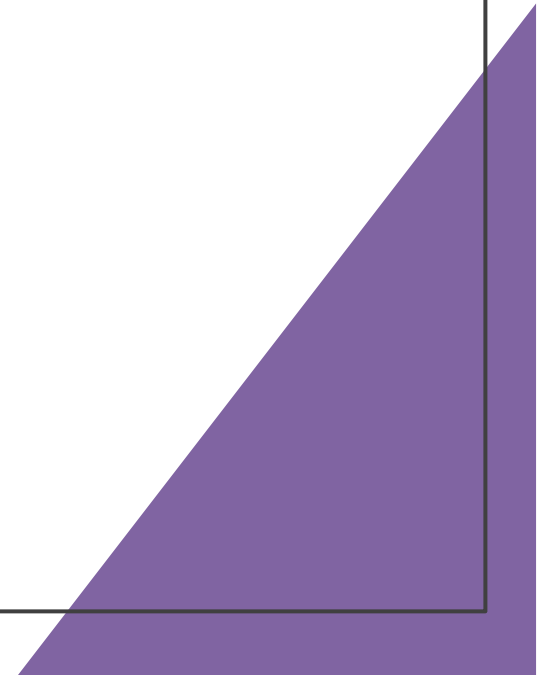
Use Framework
for legacy
Windows apps



Use Core/5+
for new apps

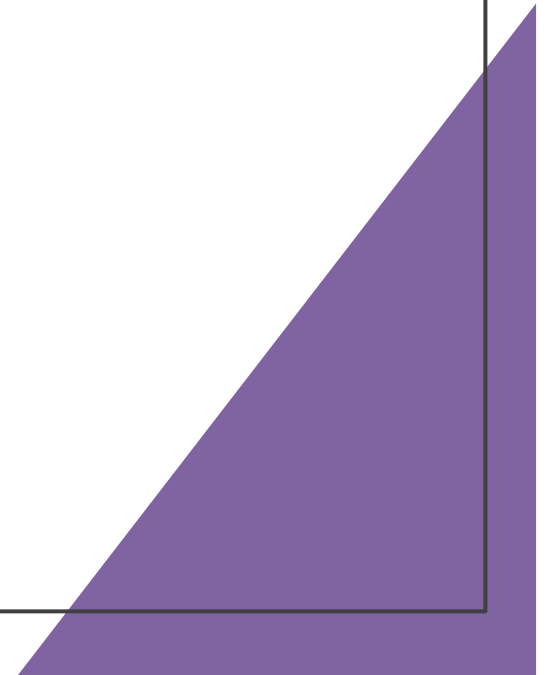
CLR Basics

Common Language Runtime manages memory, execution, security, and exceptions.



.NET Class Libraries

Rich set of libraries for I/O, networking,
collections, data, UI.

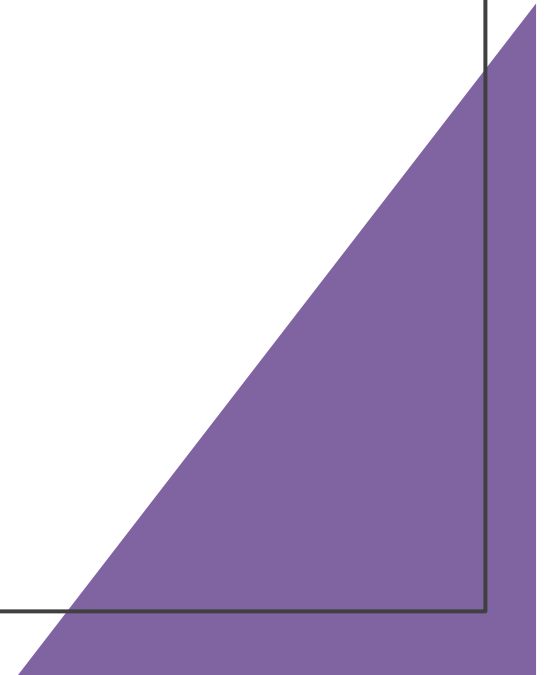


.NET Applications

Build desktop, web, mobile, cloud, microservices.

Visual Studio with .NET

Integrated development: design, code,
debug, deploy.



Summary of Visual Studio & .NET

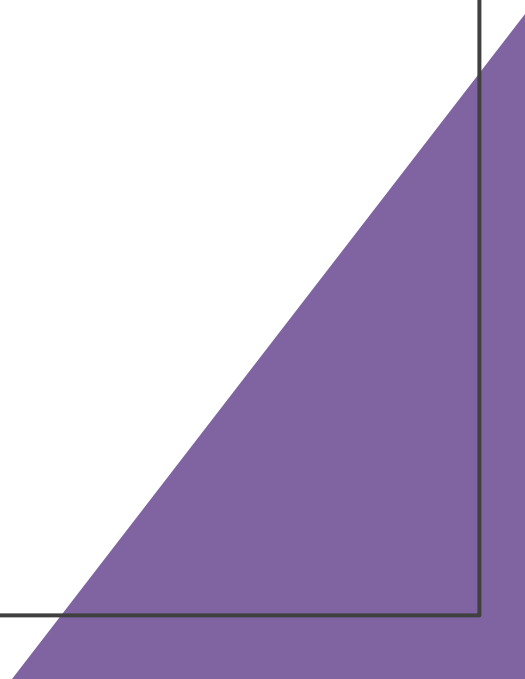
IDE usage

Debugging

Git
integration

.NET platform
basics

Q&A



Introduction to C#

Modern, object-
oriented
programming
language

Developed by
Microsoft for .NET
platform

Used for desktop,
web, mobile, and
game development

Operators and Expressions Overview

Used to perform
operations on
variables and
values

Examples:
arithmetic,
assignment,
comparison

Increment Operators: i++ vs ++i

i++ (post-increment): Returns the value, then increments

++i (pre-increment): Increments first, then returns the value

C# Programming Constructs

Sequence

Selection (if, switch)

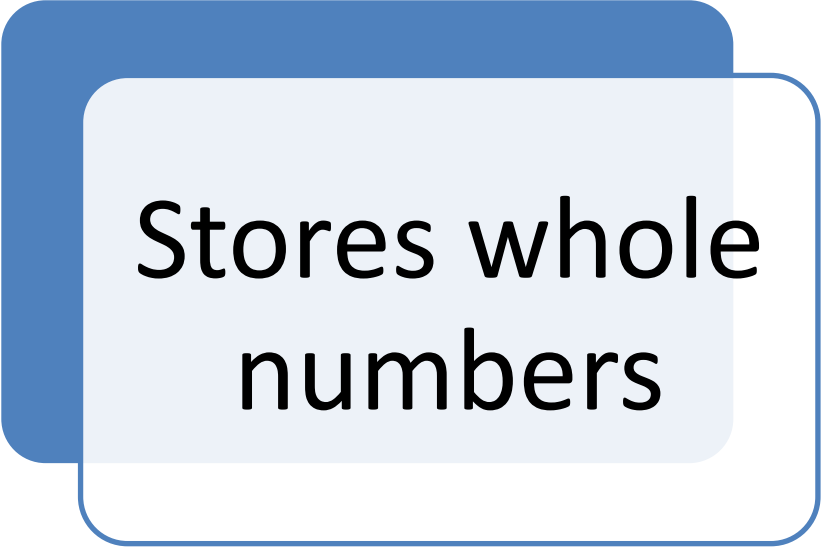
Iteration (for, while, do-while)

Data Types Overview

Value types vs
Reference types

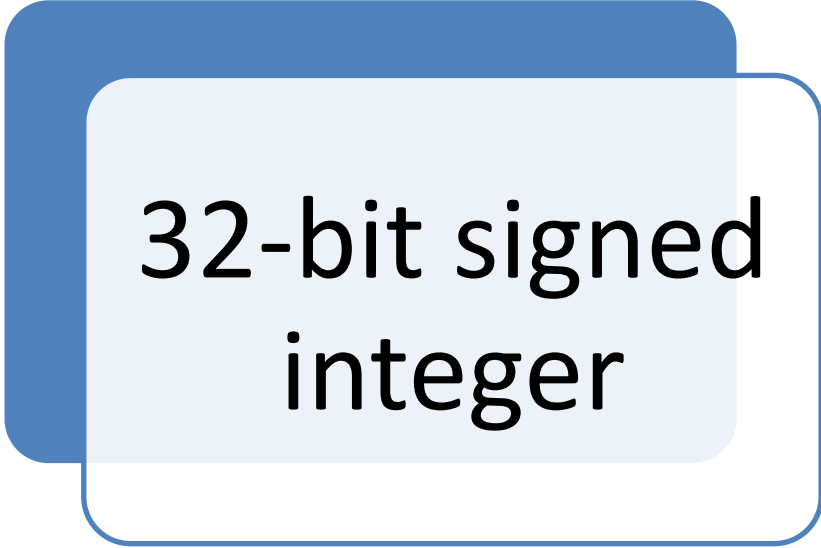
Examples: int,
double, float,
decimal, char,
string, array

The int Data Type



Stores whole
numbers

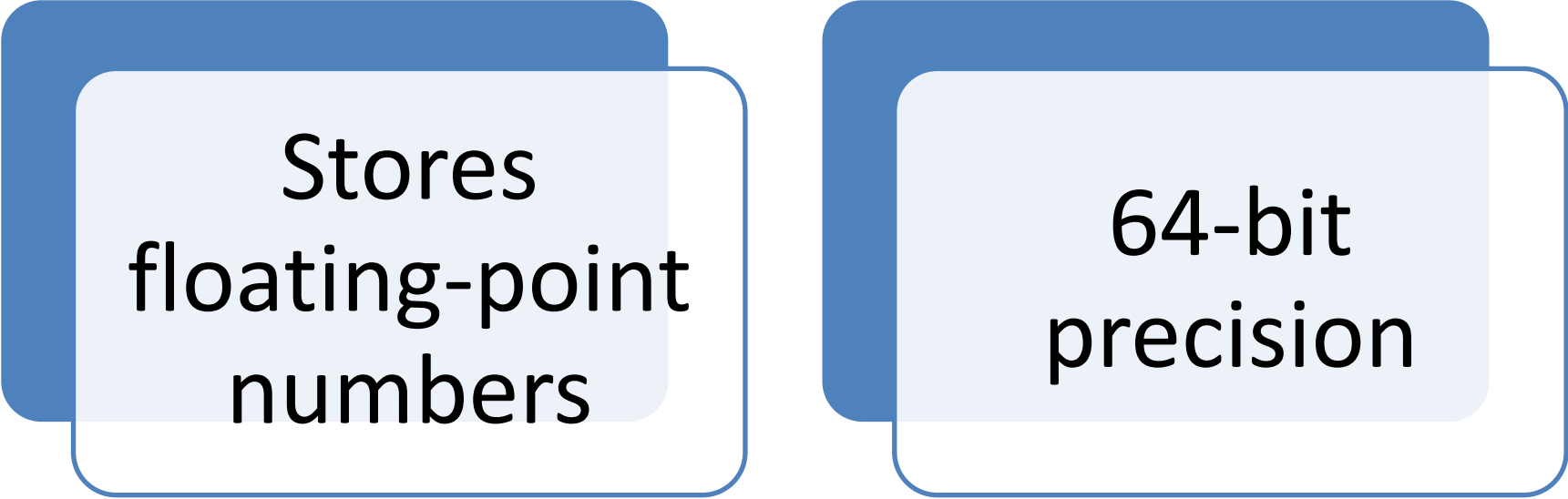
The diagram consists of two overlapping rounded rectangles. The front rectangle is light blue with a thin blue border and contains the text 'Stores whole numbers'. The back rectangle is a darker blue and is partially visible behind the first one.



32-bit signed
integer

The diagram consists of two overlapping rounded rectangles. The front rectangle is light blue with a thin blue border and contains the text '32-bit signed integer'. The back rectangle is a darker blue and is partially visible behind the first one.

The double Data Type



The diagram consists of two identical light blue rounded rectangular boxes with dark blue borders, positioned side-by-side. Each box contains a dark blue header bar at the top and a light blue body. The left box contains the text 'Stores floating-point numbers' and the right box contains '64-bit precision'.

Stores
floating-point
numbers

64-bit
precision

The decimal Data Type

High precision
decimal
numbers

Used for
financial
calculations

The float Data Type

Single
precision
floating point

Less precise
than double

Arrays in C#

Collection of
elements of
the same type

Fixed size
after creation

Example
syntax

The char Data Type

Represents a
single Unicode
character

Uses single
quotes 'a'

The string Data Type

Sequence of
characters

String is
internally an
array of chars

Uses double
quotes
"Hello"

Const vs Readonly

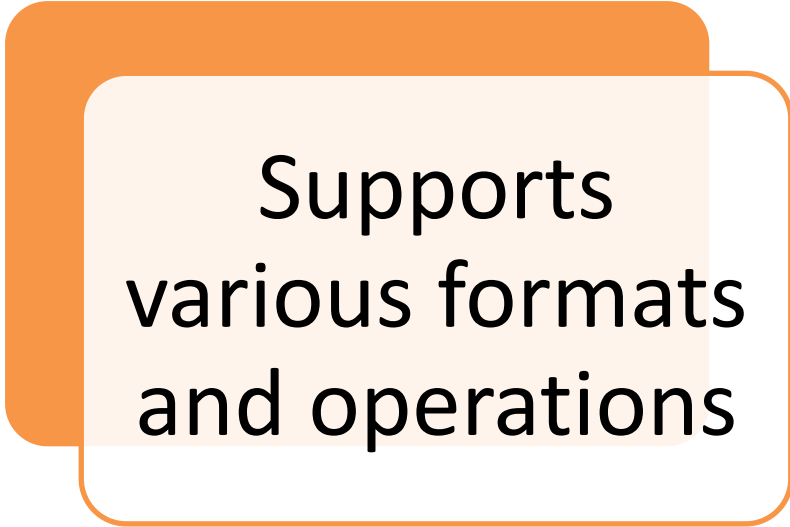
const: compile-time constant, value cannot change

readonly: runtime constant, set only once (constructor or declaration)

DateTime Basics



Represents
date and time



Supports
various formats
and operations

DateTimeOffset

Includes date,
time, and offset
from UTC

Useful for
global time
zone handling

Character Encoding Basics

ASCII: 7-bit encoding,
supports 128
characters

UTF-8: variable-length
encoding, supports all
Unicode characters

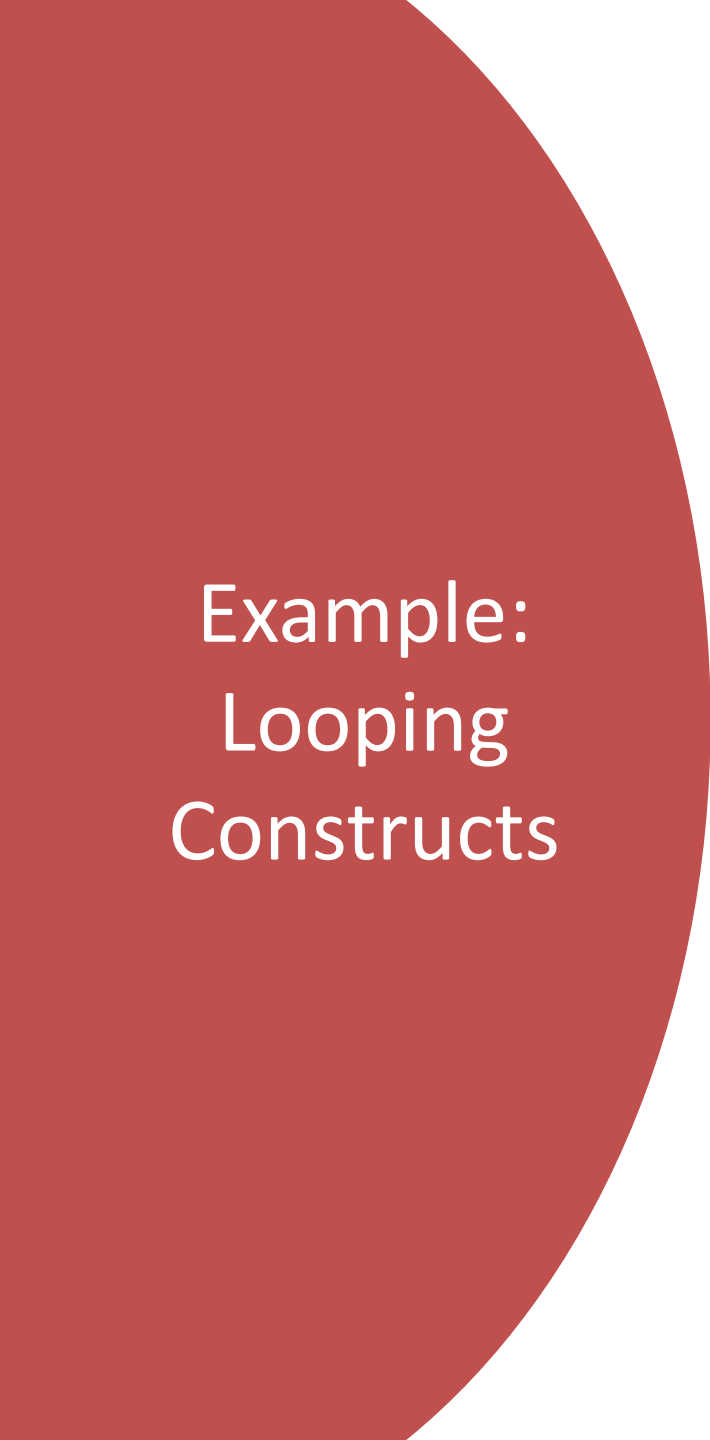
A large red circle is positioned on the left side of the slide, partially overlapping the text area.

Example: Using Operators and Expressions

```
int a = 5;  
int b = ++a; // pre-increment  
int c = b++; // post-increment
```

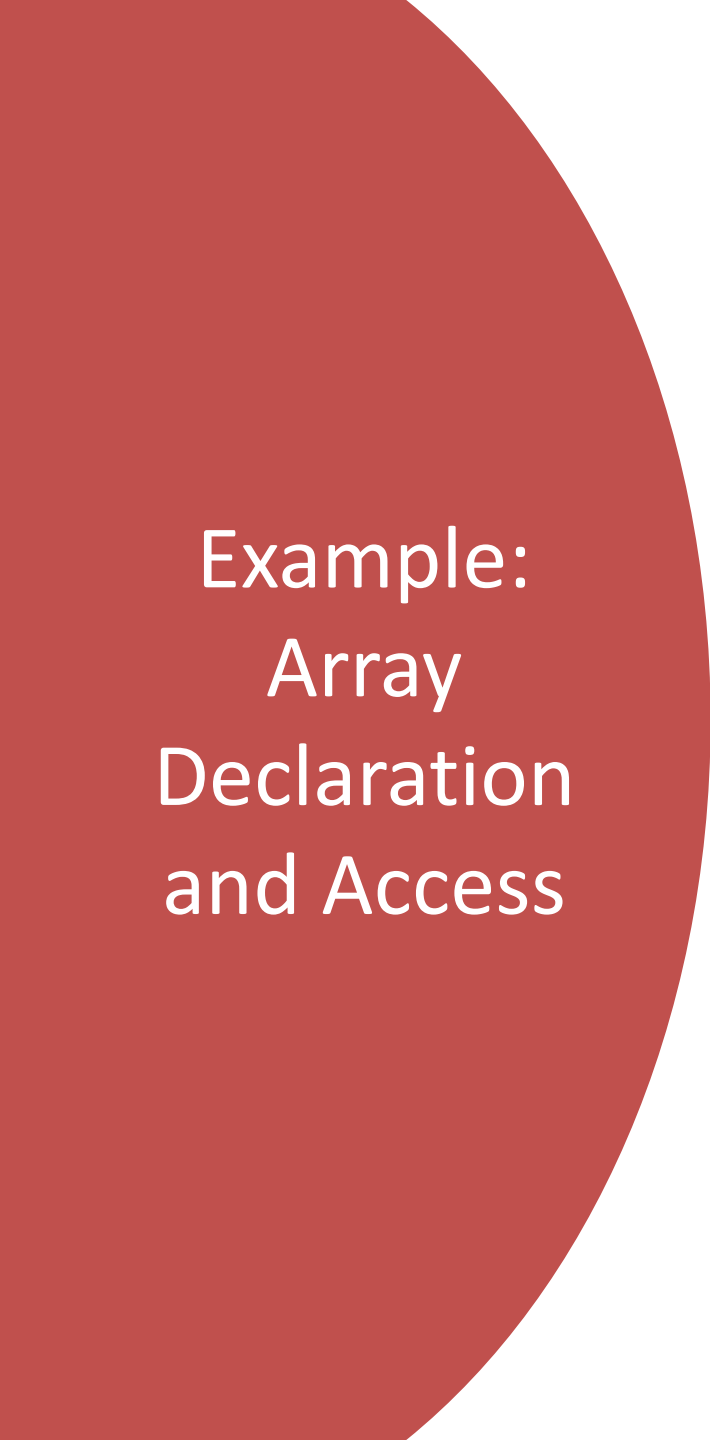
Example: Control Flow

```
if (a > 5)
{
    Console.WriteLine("Greater than 5");
}
else
{
    Console.WriteLine("5 or less");
}
```

A large red circle is positioned on the left side of the slide, partially cut off by the edge.

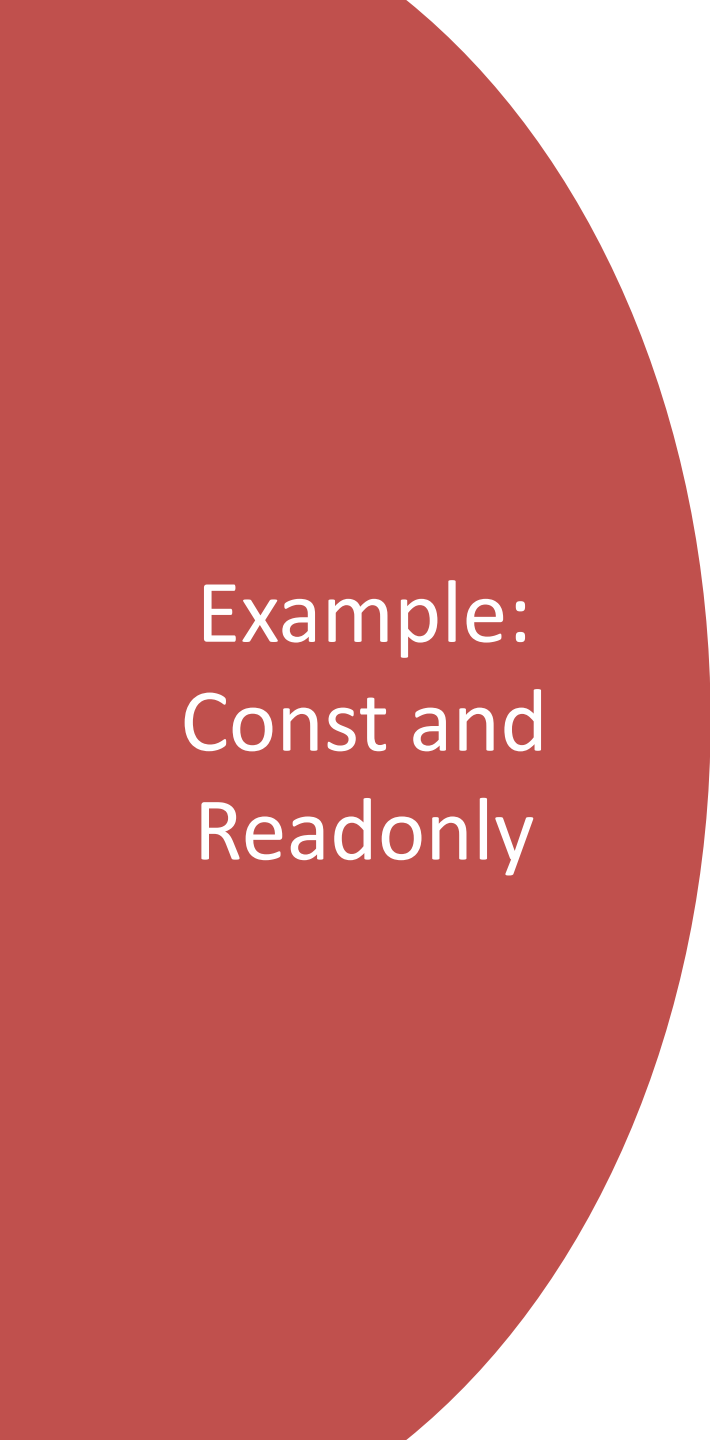
Example: Looping Constructs

```
for (int i = 0; i < 5; i++)  
{  
    Console.WriteLine(i);  
}
```

A large red circle is positioned on the left side of the slide, partially cut off by the edge.

Example: Array Declaration and Access

```
int[] numbers = new int[3] {1, 2, 3};  
Console.WriteLine(numbers[0]);
```

A large red circle is positioned on the left side of the slide, partially overlapping the text area.

Example: Const and Readonly

```
const int MaxValue = 100;  
readonly int MinValue;  
public MyClass()  
{  
    MinValue = 0;  
}
```

Summary of C# Basics

C# provides powerful constructs for programming

Understanding data types and operators is fundamental

Control flow and arrays allow building complex logic

Q&A